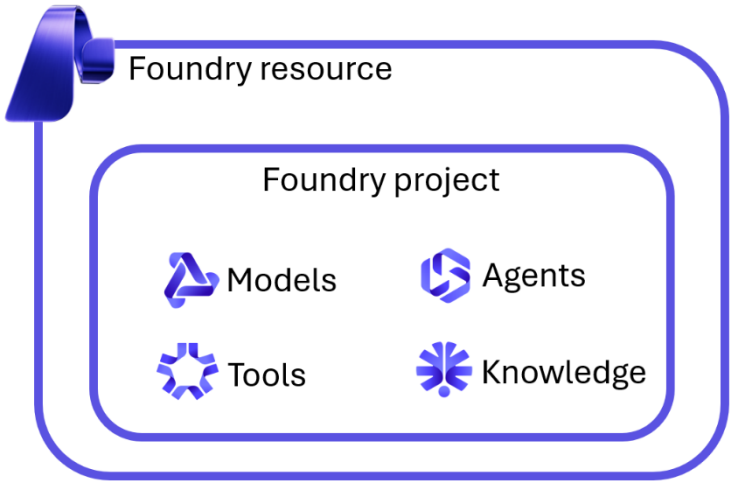
AI-901 Azure AI Fundamentals



Contents

AI-901 vs AI-103 in One Sentence 4
AI-901 "I Do This with This Tool" Chart 4
AI-901 Concepts as Modern Questions 6
Generative AI 6
Language 6
Speech 6
Vision 6
Responsible AI 7
Overlap Between AI-901 and AI-103 7
The Topics That Feel Repetitive 8
Your SQL Execution Plan Analogy? 9
Short Answer 9
Bad Plan 10
Better Plan 10
Best Plan 11
The Student Exam Shortcut 11

T Notes 12
Lab List 13
MCP 13
The Big Picture 14
AI Agent 14
MCP (Model Context Protocol) 15
MCP Client 16
MCP Server 16
Foundry Resource 17
Azure Language in Foundry Tools 18
Azure Language MCP Server 19
Entity Recognition 20
Sentiment Analysis 20
Language Detection 20
Comparing Agent vs Tool 20
Agent 20
Tool 21
SQL Execution Plan Analogy 21
Expensive 22
Better 22
Best 22
AI-901 Exam Shortcut 23
Who is thinking? 23
Who owns the standard? 23
Who asks for capability? 23
Who exposes capability? 23
Who performs deterministic NLP? 24
Where do Azure AI resources live? 24
Mechanic View of AI 25
Text 25
Images 26

Audio 27
What Do Models Actually Do? 27
Your SQL Execution Plan Analogy 28
The "Tool First" Design Pattern 30
Time 30
Math 30
Company Documents 31
Sentiment 31
My Current "Mechanic's Decision Tree" 31
The 2026 Mental Model 32
Field-Reference AI-901 Topics 33
Model Deployments Agentic API MCP 34
Loosely Understanding: An internal representation Service-specific units Processing concepts 36
AI Execution Plans 36
AI-901 One-Page Study Guide 52
AI-901 Exam Cheat Sheet 64
Perfect & Nice Prompt: 66
To Get AI-901 Practice Quiz (25 Questions) 66
Exam-Day Thinking Framework 74
"What is the lowest-cost component that can reliably answer the question?" 76
Reference 78

Prompt

Hey Co - Summarize Course AI-901T00-A: Introduction to AI similar to the AI 103 pdf, note overlap or repetitive topics in both courses.

Short, direct, for students & ideally "I do this with this tool" task to model type.

https://learn.microsoft.com/en-us/training/courses/ai-901t00

concepts as questions using current terminology:

https://www.youtube.com/watch?v=GtJwzvjzmmg

If I approach design like SQL Execution Plan, where certain commands/tools - order of - produce results better because the PC (RAM, CPU, hard drive C.R.U.D Ops - other) "like" or process the math that does the reasoning, transforms, vectoring

& billing - better:

- does that translate to model & its features choice?

Hey Tim,

AI-901 vs AI-103 in One Sentence

AI-901 = "What is AI and which Azure tool would I use?"
AI-103 = "Build it, secure it, connect it, govern it, and pay for it."

Think:

Course	Student Question
AI-901	"What wrench d I grab?"
AI-103	"How do I rebuild the engine?"

AI-901 is fundamentals. AI-103 is implementation and architecture.

AI-901 "I Do This with This Tool" Chart

I Need To...	Model Type	Azure Tool
Chat with AI	LLM / Generative AI	Azure OpenAI / Foundry
Ask questions about company docs	RAG + LLM	AI Search + OpenAI
Classify sentiment	Language AI	Azure AI Language
Extract entities from text	NLP	Azure AI Language
Convert speech to text	Speech Model	Azure AI Speech
Read text aloud	Speech Synthesis	Azure AI Speech
Describe an image	Vision Model	Azure AI Vision

I Need To...	Model Type	Azure Tool
Generate an image	Image Generation Model	DALL-E
Read invoices/forms	Document Intelligence	Azure AI Document Processing
Build an autonomous helper	Agent	Foundry Agent Service
Control risk	Safety Models	Azure AI Content Safety

This mapping appears in both courses repeatedly. AI-901 introduces it, AI-103 builds it.

AI-901 Concepts as Modern Questions

Instead of the older certification wording:

Generative AI

How does an LLM generate responses?

What is a prompt?

What is a system prompt?

When should I use RAG versus a larger context window?

What is an AI agent?

Language

How can AI detect sentiment?

How can AI extract names, places, or organizations?

How can AI summarize large amounts of text?

Speech

How does speech-to-text work?

How does text-to-speech work?

How do real-time voice agents work?

Vision

How can AI understand images?

How can AI detect objects?

How does OCR extract text from images?

Responsible AI

Should the AI answer this?

How do we prevent harmful outputs?

How do we add guardrails?

These questions show up again in AI-103, just at a deeper level. turn1search2

Overlap Between AI-901 and AI-103

About **70-80% of the concepts overlap**.

opic	AI-901	AI-103
Generative AI	✅ Intro	✅ Build apps
Agents	✅ Intro	✅ Build agents
Prompts	✅ Intro	✅ Engineering
RAG	✅ Concept	✅ Implement
Vision	✅ Concept	✅ Code
Speech	✅ Concept	✅ Code
Content Safety	✅ Concept	✅ Configure
Responsible AI	✅ Concept	✅ Governance
Foundry	✅ Tour	✅ Daily usage

Your AI-103 notes already identified the repeated pattern:

Project

↓

Model

↓

Prompt

↓

Optional Data

↓

Optional Tools

↓

Response

AI-901 teaches the boxes.

AI-103 teaches how to wire the boxes together.

The Topics That Feel Repetitive

Students often notice:

Chat

Agent

Copiot

Assistant

RAG

Foundry

Responses API

Prompt

all sound different.

Under the hood they're usually:

Human

↓

Prompt

↓

Model

↓

Optional Tool

↓

Response

which matches your AI-103 mental model.

Your SQL Execution Plan Analogy?

Short Answer

Yes. Very much yes.

Think:

SQL World	AI World
Execution Plan	AI Architecture
Index	Vector Index
Join Strategy	Retrieval Strategy
Stored Procedure	Tool / MCP Tool
Query Optimizer	Agent Planner
CPU Cost	Token Cost
I/O Cost	Search Cost
Cache	Context Window / Memory

A DBA asks:

- Can I get the answer with an index seek instead of a table scan?

An AI architect asks:

- Can Search answer before I burn GPT tokens?

Example:

Bad Plan

User

↓

GPT-5

↓

50 page PDF

↓

Answer

Huge token bill.

Better Plan

User

↓

Search

↓

Top 3 chunks

↓

GPT-5

↓

Answer

- Less inference.
- Less cost.

- Faster.

Best Plan

User: What time is it?

↓

Python datetime.now()

↓

Answer

No model.

No tokens.

No bill.

Exactly like avoiding an expensive SQL query because deterministic code can answer. This directly lines up with your **Pre-AI Guardrail Layer** and **FinOps for AI** thinking.

The Student Exam Shortcut

If AI-901 feels huge, memorize these six questions:

1. What model type solves the problem?
2. What Azure service hosts it?
3. How does the user interact with it?
4. Does it need company data (RAG)?
5. Does it need tools/actions (Agent)?
6. How do we keep it safe (Responsible AI)?

Then AI-103 becomes:

7. **How do I build it?**
8. **How do I secure it?**
9. **How do I monitor cost?**
10. **How do I connect tools with MCP?**

That's essentially the bridge from AI-901 → AI-103.

And yes... your SQL execution-plan viewpoint is one of the better mental models I've sen for Foundry architecture:

"Can I get the answer with deterministic code, search, or a tool before I spend inference?"

That's basically AI FinOps in one sentence.

T Notes

FinOps for AI

Python Guardrails aka Deterministic code maybe done at UI or Pre-AI stage:

User Q's like what time is it

- Filter: Return Top 3
- Block known abuse
- Reject unsupported file types
- Limit prompt size
- Enforce rate limits

Discovered in Foundry Portal:

- Requests by User
- Requests by Agent
- Tokens by Agent
- Tokens by Model
- Tool Calls
- Search Calls
- Image Generations
- Speech Minutes

Performance benchmarks: Quality, safety, and throughput scores.

Input and output: The formats supported for prompts and responses.

Context: The number of tokens that can be maintained in a conversation and produced as output, and when the model was trained.

Endpoints: The API endpoints through which the model can be consumed by client applications, and whether it can be used by an agent.

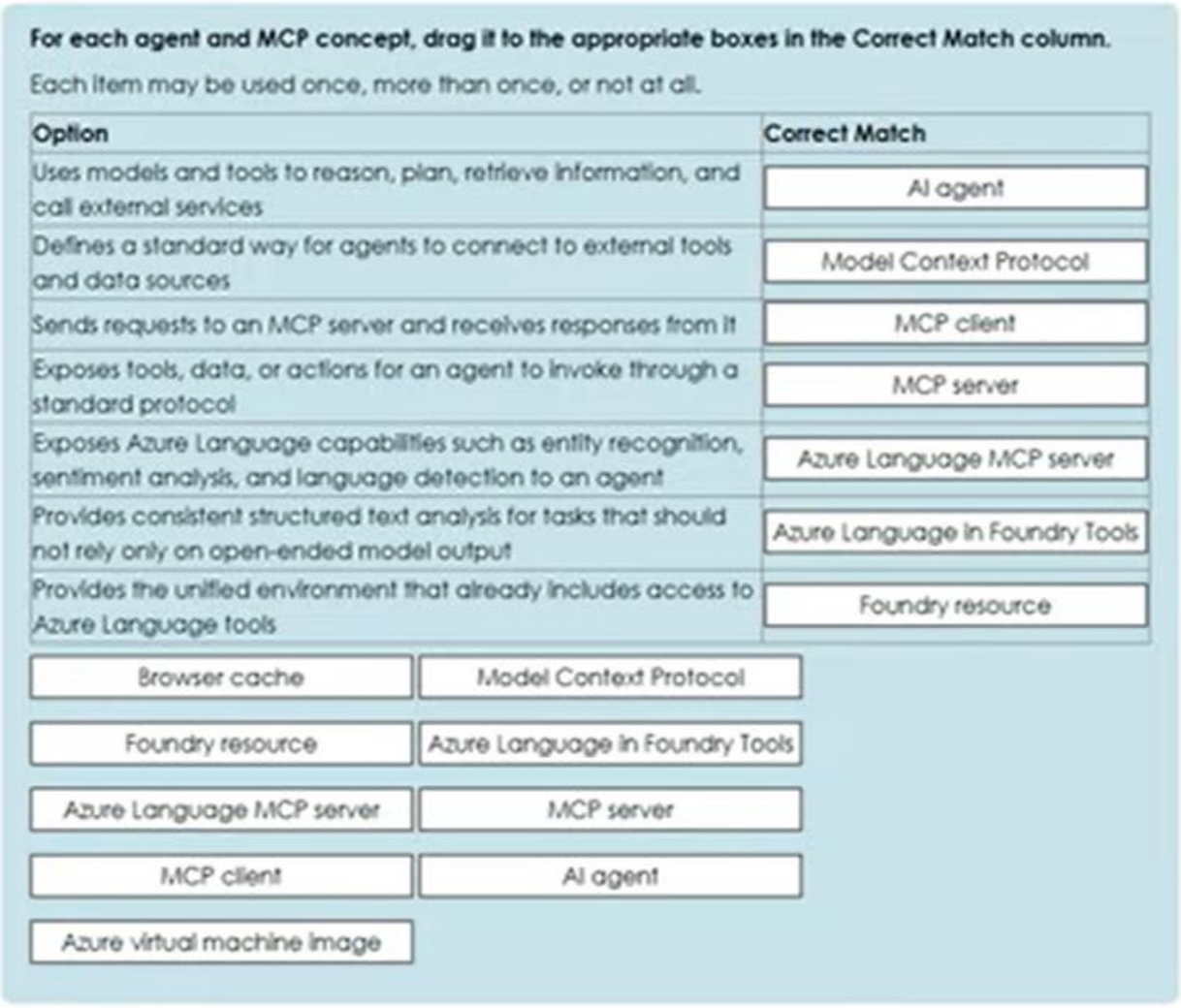
Supported features: Specific capabilities that you may require in your application scenario.

Lab List

[mslearn-ai-fundamentals/Instructions/Exercises at main · MicrosoftLearning/mslearn-ai-fundamentals]

1. Get started in Microsoft Foundry (Expected Duration 1 hours)
2. Get started with generative AI and agents in Microsoft Foundry (Expected Duration 1 hours, 30 minutes)
3. Get started with text analysis in Microsoft Foundry (Expected Duration 1 hours)
4. Get started with speech in Microsoft Foundry (Expected Duration 1 hours)
5. Get started with computer vision in Microsoft Foundry (Expected Duration 1 hours)
6. Get started with information extraction in Microsoft Foundry (Expected Duration 1 hours)

MCP



For each agent and MCP concept, drag it to the appropriate boxes in the Correct Match column.

Each item may be used once, more than once, or not at all.

Option	Correct Match
Uses models and tools to reason, plan, retrieve information, and call external services	AI agent
Defines a standard way for agents to connect to external tools and data sources	Model Context Protocol
Sends requests to an MCP server and receives responses from it	MCP client
Exposes tools, data, or actions for an agent to invoke through a standard protocol	MCP server
Exposes Azure Language capabilities such as entity recognition, sentiment analysis, and language detection to an agent	Azure Language MCP server
Provides consistent structured text analysis for tasks that should not rely only on open-ended model output	Azure Language in Foundry Tools
Provides the unified environment that already includes access to Azure Language tools	Foundry resource

Browser cache | Model Context Protocol

Foundry resource | Azure Language in Foundry Tools

Azure Language MCP server | MCP server

MCP client | AI agent

Azure virtual machine image

This image is actually a pretty good **AI-901 exam question** because it separates the generic MCP architecture from Azure-specific implementations.

The Big Picture

Think of MCP as:

USB-C for AI

Before MCP:

Agent
├── Custom API #1
├── Custom API #2
├── Custom SQL Connector
└── Custom SharePoint Connector

Every tool needed its own integration.

With MCP:

Agent
↓
MCP Client
↓
MCP Protocol
↓
MCP Server
↓
Any Tool / Data Source

One standard. Many tools.

AI Agent

Image definition:

- Uses models and tools to reason, plan, retrieve information, and call external services.

Think:

User:

- What's our Q4 revenue?

Agent:

1. Search company data

2. Run query

3. Summarize results

4. Respond

The agent is the "brain".

An agent typically contains:

- LLM
- Prompt instructions
- Memory
- Tool access
- Planning logic

MCP (Model Context Protocol)

Image definition:

Defines a standard way for agents to connect to external tools and data sources.

Think:

- HTTP = standard for web
- SQL = standard for databases
- MCP = standard for AI tools

- MCP does not contain data.
- MCP does not contain AI.
- MCP is simply the agreed language.

MCP Client

Image definition:

- Sends requests to an MCP server and receives responses.

Think:

Agent

↓

MCP Client

↓

"MCP Server:

execute search_company_docs()"

The client lives close to the agent.

In practice:

Foundry Agent

- Claude

Copilot Studio Agent

- ChatGPT

can all be MCP clients.

The client asks for:

- tools
- prompts
- resources
- actions

MCP Server

Image definition:

Exposes tools, data, or actions for an agent to invoke.

Think:

MCP Server

├── Search SharePoint

├── Query SQL

├── Create Ticket

└── Run Python

The server is not the AI.

- The server hosts capabilities.

A useful mental model:

- Agent = worker
- MCP Server = toolbox

Foundry Resource

Image definition:

- Provides the unified environment that already includes access to Azure Language tools.

Students often confuse:

- Foundry Resource

with

- Foundry Agent

A Foundry Resource is more like:

Resource Group

↓

Foundry Resource

↓

Models

↓

Agents

↓

Built-in AI Services

It's the Azure container where things live.

Azure Language in Foundry Tools

Image definition:

- Provides consistent structured text analysis for tasks that should not rely only on open-ended model output.

This is an important AI architecture concept.

Suppose you ask an LLM:

- Extract the person's name.

The model might respond:

- I believe the customer name is Tim.

Maybe.

Maybe not.

Instead use:

- Azure Language
- Named Entity Recognition

which returns structured results.

Example:

```
{
  "name": "Tim ",
```

"type": "Person"

}

Why?

Because deterministic services are usually:

- faster
- cheaper
- more reliable
- easier to audit

This aligns with your FinOps thinking:

Don't use GPT

- when a purpose-built service can do it.

Azure Language MCP Server

Image definition:

- Exposes Azure Language capabilities such as entity recognition, sentiment analysis, and language detection to an agent.

Think:

Agent

↓

MCP

↓

Azure Language MCP Server

↓

Sentiment Analysis

Examples:

Entity Recognition

Input

- Tim works at Microsoft.

Output

- Person: Tim
- Organization: Microsoft

Sentiment Analysis

Input

- The service is amazing.

Output

- Positive

Language Detection

Input

Bonjour tout le monde

Output

French

Comparing Agent vs Tool

This is where many students get confused.

Agent

Reasons.

- Should I use search?

- Should I call SQL?
- Should I summarize?

Tool

Performs an action.

- Search docs
- Get record
- Translate text

Think:

Agent = project manager

Tool = specialist worker

SQL Execution Plan Analogy

Your analogy is actually excellent.

Traditional database:

Optimizer

↓

Index Seek

↓

Table Scan

↓

Join

Agent architecture:

User Question

↓

Agent Planner

↓

Choose Tool

↓

Execute

↓

Return Result

The modern AI question becomes:

What is the cheapest path

to a trustworthy answer?

Possible execution plans:

Expensive

User

↓

GPT-5

↓

20,000 Tokens

Better

User

↓

AI Search

↓

Top 3 Chunks

↓

GPT-5

Best

User:

What time is it?

↓

Tool Call

↓

datetime.now()

No reasoning.

No vector search.

No inference.

Minimal cost.

AI-901 Exam Shortcut

When you see a question involving MCP ask:

Who is thinking?

✅ Agent

Who owns the standard?

✅ MCP

Who asks for capability?

✅ MCP Client

Who exposes capability?

✅ MCP Server

Who performs deterministic NLP?

✅ Azure Language

Where do Azure AI resources live?

✅ Foundry Resource

That simple mapping will answer most MCP-related AI-901 questions correctly.

Yes. You're basically thinking like a mechanic looking under the hood rather than a driver using the dashboard.

The simplification I'd use is:

Human Meaning

↓

Math

↓

Model Processing

↓

Math

↓

Human Meaning

The model never truly sees:

- text
- images
- audio
- video

the way humans do.

Everything eventually becomes:

Vectors

Matrices

Probabilities

Weights

Relationships

Distances

and then gets transformed back into something humans recognize.

Mechanic View of AI

Forget the marketing terms for a minute.

An AI system is basically:

Input

↓

Encoding

↓

Math Space

↓

Reasoning / Prediction

↓

Tool Calls

↓

Output

Text

User enters:

What is MCP?

Model sees something closer to:

[0.23, -0.91, 0.77, ...]

called an embedding/vector.

Words with similar meanings end up near each other.

Dog ≈ Puppy

Car ≈ Vehicle

King ≈ Queen

are mathematically close.

Images

Image:

Cat sitting on sofa

becomes:

Pixels

↓

Features

↓

Vectors

The model learns:

Pointy ears

Whiskers

Fur pattern

Body shape

The model doesn't know "cat".

It knows:

Pattern strongly correlates

with previous cat examples.

Audio

Speech:

"Open the file"

becomes:

Sound Wave

↓

Frequency Analysis

↓

Tokens

↓

Vectors

Eventually becoming text the LLM understands.

What Do Models Actually Do?

The simplest answer:

- Predict the next best thing.

Text:

- The sky is ...

Predict:

- blue

Image:

- Draw a sunset

Predict:

- which pixels belong next

Agents:

- What should happen next?

Predict:

- Call Search Tool

or

- Call SQL Tool

Your SQL Execution Plan Analogy

This is surprisingly accurate.

Database:

Optimizer

↓

Execution Plan

↓

Indexes

↓

Result

Agent:

Prompt

↓

Planner

↓

Tool Selection

↓

Result

The AI version of an index seek might be:

Vector Search

The AI version of a stored procedure might be:

MCP Tool

The AI version of an execution plan:

Reason

↓

Search

↓

Retrieve

↓

Analyze

↓

Respond

The "Tool First" Design Pattern

As a developer, this is where cost and performance matter.

Bad:

Everything

↓

GPT-5

Expensive.

Better:

Can a tool answer it?

↓

Yes

↓

Use tool

Examples:

Time

Tool:

datetime.now()

not

LLM reasoning

Math

Python

not

LLM estimate

Company Documents

AI Search

not

Entire document into GPT

Sentiment

Azure Language

not

Prompt GPT

My Current "Mechanic's Decision Tree"

When building agents:

1. Can deterministic code do it?

    YES → Use code

2. Can a service do it?

    YES → Use service

3. Can search find it?

    YES → Search

4. Need reasoning?

    YES → Use model

5. Need external actions?

    YES → Use Agent + MCP

The 2026 Mental Model

For Foundry, MCP, Agents, Copilot Studio, OpenAI, Claude, etc.:

Model

= Thinking

Tool

= Doing

Search

= Finding

MCP

= Standardized Wiring

Agent

= Deciding Which To Use

Guardrails

= Limiting Behavior

FinOps

= Choosing Cheapest Trustworthy Path

That's probably the most useful "Dev Mechanic" summary:

Don't ask "Which model?" first. Ask "What combination of search, tools, deterministic code, and model reasoning solves the problem with the lowest cost and highest reliability?"

That's the AI equivalent of looking at a SQL execution plan before blindly throwing more CPU at the server.

Field-Reference AI-901 Topics

1. **Need conversational AI?** → Use an **LLM/Generative AI model**.
   Why: Generates natural language responses.
2. **Need company-document chat?** → Use **RAG + Azure AI Search + LLM**.
   Why: Grounds answers in your data.
3. **Need sentiment detection?** → Use **Azure AI Language**.
   Why: Built specifically for text sentiment.
4. **Need entity extraction?** → Use **Named Entity Recognition (NER)**.
   Why: Structured identification of people, places, organizations.
5. **Need speech-to-text?** → Use **Azure AI Speech**.
   Why: Converts spoken audio into text.
6. **Need text-to-speech?** → Use **Azure AI Speech**.
   Why: Generates spoken audio.
7. **Need image analysis?** → Use **Azure AI Vision**.
   Why: Understands image content.
8. **Need OCR?** → Use **Vision OCR / Document Intelligence**.
   Why: Extracts text from images and documents.
9. **Need invoice or form extraction?** → Use **Document Intelligence**.
   Why: Returns structured fields.
10. **Need image generation?** → Use an **image generation model**.
    Why: Creates new images from prompts.
11. **Need an AI to choose tools?** → Use an **Agent**.
    Why: Plans and invokes actions.
12. **Need tool connectivity?** → Use **MCP**.
    Why: Standard protocol between agents and tools.
13. **Need deterministic language analysis?** → Use **Azure Language**, not an LLM prompt.
    Why: More consistent and cheaper.
14. **Need safety filtering?** → Use **Content Safety**.
    Why: Applies AI guardrails.
15. **Need semantic document retrieval?** → Use **Vector Search**.
    Why: Finds meaning, not just keywords.
16. **Need structured search?** → Use **Keyword/Metadata Search**.
    Why: Faster for exact matches.
17. **Need a predictable result?** → Use a **tool or API first**.
    Why: Deterministic beats probabilistic.
18. **Need model creativity?** → Increase **temperature**.
    Why: More variation in outputs.
19. **Need consistent answers?** → Lower **temperature**.
    Why: Less randomness.
20. **Need AI governance?** → Apply **Responsible AI principles**.
    Why: Fairness, transparency, privacy, reliability.

AI-901 Exam Shortcut

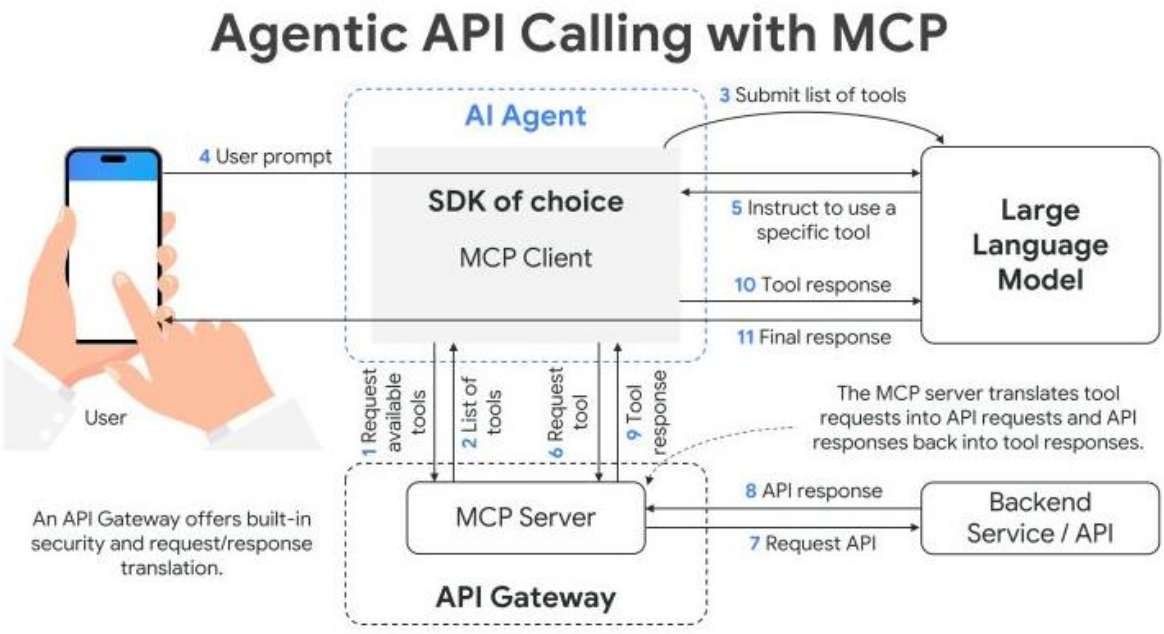
For almost every scenario question, ask:

1. What is the task?
2. What model type fits?
3. What Azure service provides it?
4. Can a deterministic service do it cheaper?
5. Does it need company data (RAG)?
6. Does it need actions/tools (Agent)?
7. Does it need guardrails (Content Safety)?

That's essentially 70-80% of AI-901 and lines up with the newer Foundry/MCP terminology now appearing in training content.

Model Deployments Agentic API MCP

Calling a published agent versus calling a model deployment comes down to level of autonomy versus raw inference control.



Calling a Model Deployment

- **What it is:** You send raw text prompts (via an API like the Responses API or OpenAI-compatible endpoint) directly to a hosted foundation model (e.g., Azure OpenAI, DeepSeek, or Llama) configured with specific quotas and SKUs. [1, 2]

- **Control:** You manage the entire orchestration loop—system prompts, memory, multi-turn conversation state, RAG (retrieval-augmented generation), and tool/API calling logic in your own application code. [1, 2]
- **Best used when:** You want maximum flexibility, custom state management, or already handle agentic workflows outside of a managed service. [1]

Calling a Published Agent (Agent Application / Hosted Agent)

- **What it is:** You invoke a pre-configured or code-wrapped agent unit (created in Microsoft Foundry or similar platforms) that encapsulates instructions, an assigned model deployment, attached tools, identity, and built-in memory/planning loops. [1, 2]
- **Control:** The platform or container manages the orchestration, execution loops, tool dispatch, and session governance; your app interacts with a durable, addressable service entry point. [1, 2, 3]
- **Best used when:** You want to delegate multi-step reasoning, automated tool/code execution, and channel integrations (like Microsoft Teams or M365 Copilot) to a managed agent runtime.

To call a published agent using the Azure AI Projects / Microsoft Foundry Python SDK, you use **project_client.get_openai_client(agent_name=myAgent)** (or pass agent_name when calling get_openai_client) combined with **openai_client.responses.create(...)**. [1, 2]

Breakdown of the Methods

- **project_client.agents.get(agent_name=myAgent)**: Used to retrieve or inspect the metadata/definition of an agent object within the project, not to execute or send inference requests to it.
- **project_client.get_openai_client(agent_name="my-agent")**: Used to obtain an authenticated OpenAI-compatible client bound to your specific hosted agent.
- **openai_client.responses.create(...)**: Used to actually invoke/call the agent and run a response or conversation turn.

This is a Swing at the fences, not sure where it lands:

Loosely Understanding: An internal representation Service-specific units Processing concepts

As it applies to:

Tokenization - Training | Microsoft Learn

Things that Constitute Tokens (billable) & Re Usable for A Defined Purpose

aka: Exotic Data Math (an incomplete growing vocabulary area) Frequency Inverse Inference Bag-of-words TextRank Recognition Synthesis MFCCs Predict Statistical Context Adaptation Grapheme-to-phoneme Prosody Transformers Classification Detection Semantic segmentation Notifications and alerts

- Filters Embeddings Attention Diffusion Extraction RAG (& MCP) Grounded Sentiment Entity

Ultimately calling the published agent vs calling a model deployment:
project*client.agents.get(agent*name=myAgent) project*client.get*openai*client()*
*openai*client.responses.create

AI Execution Plans

Tim, I think this lands surprisingly well.

What you're circling around is something I've started calling:

"AI Execution Plans"

Just like SQL has internal operators (Seek, Scan, Join, Sort, Hash Match), AI workloads have internal operators and billing units that most students never see.

Layer 1: What Are Tokens Really?

Students learn:

Token = billable unit

That's true, but incomplete.

Think:

Character

↓

Word Pieces

↓

Tokens

↓

Embeddings

↓

Math

Example:

Microsoft develops AI

might become:

["Micro","soft","develops","AI"]

Those token IDs become vectors.

The model does not reason on words.

The model reasons on **numbers representing words**.

Layer 2: Service-Specific Units

Not every AI service bills by tokens.

Service	Internal Unit
GPT Models	Tokens
AI Search	Queries / Documents
Vision	Images
Speech	Audio Seconds

Service	Internal Unit
Translation	Characters
Embeddings	Tokens Processed
Document Intelligence	Pages
Agent Service	Underlying Service Usage

The AI-901 course mostly hides this distinction.

The architect eventually learns:

Same Question

↓

Different Services

↓

Different Billing Units

Layer 3: The Vocabulary Behind The Magic

These are not random buzzwords.

They represent distinct processing stages.

Frequency

How often a word appears.

cloud

cloud

cloud

cloud

database

Frequency:

cloud = 4

database = 1

Used heavily in search engines.

Inverse Document Frequency (IDF)

Common words become less valuable.

the

and

is

occur everywhere.

vector

embedding

transformer

occur less often.

Therefore:

Rare terms

=

Higher importance

Bag of Words

Old school NLP.

Treat document as:

Word Counts

Ignore order.

I love AI

AI loves me

look similar.

Modern LLMs largely moved beyond this.

TextRank

Think:

PageRank

for Sentences

Often used in summarization.

Find important sentences.

Build summary.

Speech Concepts

Recognition

Speech

↓

Text

Azure Speech-to-Text.

Synthesis

Text

↓

Speech

Azure Text-to-Speech.

MFCCs

(Mel Frequency Cepstral Coefficients)

Before transformers dominated speech:

Audio

↓

Frequency Features

↓

Model

MFCCs are engineered audio features.

Think:

Audio Fingerprints

for sound recognition.

Grapheme-to-Phoneme

Text

↓

Pronunciation

Example:

Azure

becomes pronunciation instructions.

Used before speech synthesis.

Prosody

How speech sounds:

Pitch

Rhythm

Stress

Pauses

Without prosody:

robot voice

With prosody:

natural voice

Machine Learning Concepts

Classification

Predict category:

Spam

Not Spam

Prediction

Predict value:

House Price

$450,000

Detection

Find something exists.

Object Detection

Face Detection

Fraud Detection

Semantic Segmentation

Vision on steroids.

Instead of:

Car detected

you get:

Every Pixel

Belongs To Something

Example:

Road

Building

Tree

Sky

Vehicle

pixel by pixel.

Used in autonomous vehicles.

LLM Concepts

Context

The information currently available.

Prompt

+

Conversation

+

Retrieved Data

=

Context

No context.

No good answer.

Attention

The breakthrough.

Instead of reading:

left → right

the model can determine:

Which words matter most?

Attention is why Transformers changed everything.

Transformers

Architecture introduced in:

Attention Is All You Need

They power:

- GPT
- Claude
- Gemini
- Copilot models
- Azure OpenAI models

Think:

Transformer

=

Massive Pattern Finder

Inference

This is a BIG one.

Training:

Learning Patterns

Inference:

Using Learned Patterns

Every time you send:

What's the capital of Florida?

the model performs inference.

Inference is usually what you're paying for.

Embeddings

This is where your SQL analogy gets very strong.

Document:

How to reset password

becomes:

[0.345

0.891

0.104

...

]

Thousands of dimensions.

Now similarity becomes math.

Question

↓

Embedding

↓

Vector Search

↓

Closest Match

Not keyword matching.

Distance matching.

RAG

Retrieval Augmented Generation

Execution plan:

Question

↓

Embedding

↓

Vector Search

↓

Top Chunks

↓

Prompt

↓

LLM

SQL analogy:

Index Seek

↓

Rows Found

↓

Query Output

Grounded Responses

Without grounding:

Model Memory

With grounding:

Model

+

Company Data

Grounded answers typically hallucinate less.

Entity Extraction

Tim works at Microsoft in Florida

becomes:

{

"Person":"Tim",

"Organization":"Microsoft",

"Location":"Florida"

}

Deterministic.

Structured.

Often cheaper than GPT.

MCP

This is where the vocabulary starts looking like query-planning.

MCP allows:

Model

↓

Discover Tool

↓

Call Tool

↓

Get Result

The model doesn't execute actions itself.

It requests actions.

Exactly like:

SELECT *

FROM Sales

doesn't retrieve rows.

The SQL engine retrieves rows.

The query merely requests them.

Published Agent vs Direct Model Call

This may be the most important distinction in Foundry today.

Direct Model

openai_client.responses.create(...)

Execution Plan:

Prompt

↓

Model

↓

Response

Simple.

Cheap.

Fast.

No orchestration.

Agent

project_client.agents.get(...)

Execution Plan:

Prompt

↓

Agent Instructions

↓

Memory

↓

Tool Discovery

↓

MCP Servers

↓

Search

↓

Model

↓

Response

More capable.

More moving parts.

Often more cost.

The Mental Model I Think You're Building

CPU

RAM

Disk

Network

became

Tokens

Context

Embeddings

Search

Tools

Inference

and

SQL Execution Plan

became

Agent Execution Plan

Where the architect asks:

Can deterministic code answer?

If not...

Can Search answer?

If not...

Can a tool answer?

If not...

Can a smaller model answer?

If not...

Use the expensive model.

That mindset is essentially **AI FinOps + Agent Design + Responsible Cost Management** rolled into one mental framework, and honestly it's one of the cleaner ways I've seen to explain Foundry architecture to students coming from infrastructure, database, or developer backgrounds.

AI-901 One-Page Study Guide

Thinking About AI Like a SQL Execution Plan

Core Idea

Traditional Computing:

CPU + RAM + Disk + Network

↓

Execution Plan

Modern AI:

Tokens + Context + Embeddings + Search + Tools

↓

Agent Execution Plan

The goal is the same:

Find the cheapest, fastest, and most trustworthy path to an answer.

AI Processing Vocabulary

Tokens

The basic unit most LLMs process and bill on.

Text

↓

Tokenization

↓

Tokens

↓

Vectors

↓

Inference

Example:

"Microsoft develops AI"

may become:

["Micro", "soft", "develops", "AI"]

Inference

Using a trained model to generate an answer.

Prompt

↓

Model

↓

Response

Training = Learning

Inference = Using What Was Learned

Context

Everything available to the model right now.

System Prompt

+ Conversation

+ Retrieved Data

+ User Prompt

Context

Better context usually produces better answers.

Attention

The mechanism that lets a model determine:

What matters most?

Attention is the breakthrough that made modern LLMs possible.

Transformers

The architecture behind:

- GPT
- Claude
- Gemini
- Copilot
- Azure OpenAI Models

Think:

Massive Pattern Matching Engine

Search & Retrieval Concepts

Frequency

How often a word appears.

AI AI AI Database

Frequency:

AI = 3

Database = 1

Inverse Document Frequency (IDF)

Rare words carry more meaning.

the

and

is

Less useful

transformer

embedding

vector

More valuable

Bag-of-Words

Old NLP technique.

Treats a document as:

Word Counts

Ignores word order.

TextRank

Like PageRank for text.

Used to identify important sentences for summaries.

Embeddings & Vector Search

Embeddings

Convert text into numbers.

Reset Password

becomes

[0.245, 0.812, ...]

Thousands of dimensions.

This makes similarity measurable.

Vector Search

Instead of:

Keyword Search

Use:

Meaning Search

Question

↓

Embedding

↓

Vector Search

↓

Closest Content

RAG (Retrieval-Augmented Generation)

The modern enterprise pattern.

User Question

↓

Embedding

↓

Vector Search

↓

Relevant Documents

↓

LLM

↓

Response

Think:

Index Seek for AI

Grounding

Adding trusted business data to the prompt.

Without Grounding:

Model Memory Only

With Grounding:

Model

+

Company Data

Result:

- Better accuracy
- Fewer hallucinations

Azure AI Language Concepts

Sentiment Analysis

"I love this course."

Output:

Positive

Entity Extraction

Tim works for Microsoft in Florida

Output:

{

"Person": "Tim",

"Organization": "Microsoft",

"Location": "Florida"

}

Classification

Assign a category.

Spam

Not Spam

Speech Concepts

Recognition

Speech

↓

Text

Speech-to-Text

Synthesis

Text

↓

Speech

Text-to-Speech

MFCCs

(Mel-Frequency Cepstral Coefficients)

Audio features used to represent sounds mathematically.

Think:

Audio Fingerprints

Grapheme-to-Phoneme

Text

↓

Pronunciation

Used in speech generation.

Prosody

Controls:

- Pitch
- Rhythm
- Emphasis
- Pauses

Makes speech sound natural.

Vision Concepts

Detection

Identify an object exists.

Car Found

Recognition

Determine what was found.

Car

Truck

Bicycle

Semantic Segmentation

Pixel-level understanding.

Road

Building

Tree

Vehicle

Sky

Every pixel belongs to a class.

MCP (Model Context Protocol)

Think:

USB-C for AI

Standard way to connect models to tools.

Agent

Reasons and plans.

Should I search?

Should I call a tool?

Should I summarize?

MCP Client

Requests capabilities.

Agent

↓

Client

↓

Server

MCP Server

Exposes capabilities.

Search

SQL

SharePoint

Functions

Tool

Performs work.

Query Database

Search Documents

Send Email

Run Code

Foundry Architecture

Direct Model Call

openai_client.responses.create(...)

Execution Plan:

Prompt

↓

Model

↓

Response

Best for:

- Chat
- Summarization
- Content Generation

Agent Call

project_client.agents.get(...)

Execution Plan:

Prompt

↓

Agent Instructions

↓

Memory

↓

Tool Selection

↓

MCP Servers

↓

Model

↓

Response

Best for:

- Multi-step tasks
- Search
- Business workflows
- Tool orchestration

AI-901 Exam Cheat Sheet

Ask these questions:

1. What solves the problem?

Need	Tool
Chat	Azure OpenAI
Company Documents	RAG
Images	AI Vision
OCR	Vision

Need	Tool
Speech	AI Speech
Text Analysis	AI Language
Agents	Foundry Agent

---

2. Can a deterministic tool do it?

Prefer:

Entity Extraction → Language

OCR → Vision

Current Time → Tool Call

Invoice Parsing → Document Intelligence

before using an expensive LLM.

---

3. Who does what?

Component	Responsibility
Agent	Thinks
MCP Client	Requests
MCP Protocol	Standard
MCP Server	Exposes capabilities
Tool	Executes actions
Model	Generates responses

Ultimate AI Architect Question

Can deterministic code answer?

If not...

Can Search answer?

If not...

Can a Tool answer?

If not...

Can a Small Model answer?

If not...

Use the Large Model.

That's AI FinOps, Agent Design, RAG, MCP, and AI-901 thinking in one framework.

Perfect & Nice Prompt:

In the Learning Path links from this Course are module assessment questions. Put together a 25 question quiz. Using MS Learn as a book & exam reference so students can easily map it out. The Course (book) Course AI-901T00-A: Introduction to AI in Azure - Training | Microsoft Learn - the TOC tool to follow along: MS Training Navigator - Chrome Web Store – thanks

Absolutely. For AI-901, I would structure the quiz around the actual Microsoft Learn flow so students can use the course as an open-book reference and navigate directly with your **MS Training Navigator** extension.

To Get AI-901 Practice Quiz (25 Questions)

Module 1 - AI Workloads and Responsible AI

1.

You need a system that can answer employee questions using company policies stored in PDF files.

Which AI workload is MOST appropriate?

A. Computer Vision
B. Conversational AI with Retrieval
C. Speech Recognition
D. Forecasting

Answer: B

2.

A retailer wants to predict next month's product demand using previous sales data.

What type of AI workload is this?

A. Computer Vision
B. Natural Language Processing
C. Predictive Analytics / Machine Learning
D. Generative AI

Answer: C

3.

Which Responsible AI principle focuses on ensuring similar outcomes for all users?

A. Reliability and Safety
B. Fairness
C. Transparency
D. Accountability

Answer: B

4.

An AI system recommends loan approvals but cannot explain why.

Which Responsible AI principle is MOST affected?

A. Inclusiveness
B. Transparency
C. Privacy
D. Reliability

Answer: B

Module 2 - Fundamentals of Machine Learning

5.

A model predicts whether a customer will cancel a subscription.

What type of machine learning problem is this?

A. Regression
B. Classification
C. Clustering
D. Reinforcement Learning

Answer: B

6.

A model predicts the selling price of a home.

What machine learning technique is being used?

A. Regression
B. Classification
C. Clustering
D. NLP

Answer: A

7.

A data scientist groups customers into similar purchasing behaviors without predefined labels.

What type of learning is this?

A. Supervised Learning
B. Reinforcement Learning
C. Unsupervised Learning
D. Deep Learning

Answer: C

8.

Which metric is commonly used to evaluate a classification model?

A. Accuracy
B. Mean Absolute Error
C. RMSE
D. Variance

Answer: A

Module 3 - Azure AI Fundamentals

9.

Which Azure service is primarily used to host foundation models and generative AI applications?

A. Azure SQL Database
B. Azure AI Foundry / Azure OpenAI
C. Azure Storage Account
D. Azure Monitor

Answer: B

10.

What is a foundation model?

A. A database optimized for AI workloads
B. A pre-trained model adaptable to many tasks
C. A clustering algorithm
D. A Power BI report

Answer: B

11.

Which statement best describes Generative AI?

A. It only classifies records
B. It creates new content from learned patterns
C. It stores structured data
D. It replaces databases

Answer: B

Module 4 - Computer Vision

12.

You need to extract printed text from scanned invoices.

Which capability should you use?

A. OCR
B. Sentiment Analysis
C. Speech Recognition
D. Classification

Answer: A

13.

A system identifies cars, bicycles, and pedestrians within an image.

This is an example of:

A. Object Detection
B. OCR
C. Translation
D. Regression

Answer: A

14.

A model produces:

"An image showing a red car parked near a building."

What AI capability produced this result?

A. Face Detection
B. Image Captioning

C. Classification
D. Clustering

Answer: B

Module 5 - Natural Language Processing

15.

Determine whether customer feedback is positive, neutral, or negative.

Which Azure AI Language feature performs this task?

A. Named Entity Recognition
B. Summarization
C. Sentiment Analysis
D. Translation

Answer: C

16.

Given:

Tim works for Microsoft in Florida.

Which NLP capability identifies "Tim" as a person and "Microsoft" as an organization?

A. OCR
B. Named Entity Recognition
C. Translation
D. Speech Synthesis

Answer: B

17.

A user submits text in French and the service identifies the language automatically.

Which capability is being used?

A. Language Detection
B. Summarization
C. Classification
D. OCR

Answer: A

18.

Which workload is MOST commonly associated with Large Language Models?

A. Computer Vision
B. NLP and Generative AI
C. Forecasting Only
D. Clustering

Answer: B

Module 6 - Speech AI

19.

Converting spoken words into text is called:

A. Speech Synthesis
B. OCR
C. Speech-to-Text
D. Translation

Answer: C

20.

Converting text into natural-sounding audio is called:

A. Speech Synthesis
B. Speech Detection
C. OCR
D. Classification

Answer: A

21.

A multilingual voice assistant translates speech during a meeting.

Which Azure service area is primarily involved?

A. Computer Vision
B. Azure AI Speech
C. Azure SQL
D. Azure DevOps

Answer: B

Module 7 - Generative AI & Agents

22.

What is the primary purpose of Retrieval Augmented Generation (RAG)?

A. Train a model from scratch
B. Reduce hardware requirements
C. Ground responses using external data
D. Improve image quality

Answer: C

23.

Which statement best describes an AI Agent?

A. A database with vector indexing
B. A workflow that reasons, plans, and uses tools to complete tasks
C. A speech recognition engine
D. A storage account

Answer: B

24.

In the MCP architecture, which component exposes tools and actions to an agent?

A. MCP Client
B. Agent Prompt
C. MCP Server
D. Vector Store

Answer: C

25.

Which sequence BEST represents modern Azure AI application design?

A.

User → SQL → Storage

B.

User → Model → Delete Data

C.

User → Agent → Tool/Search → Model → Response

D.

User → PowerPoint → Agent

Answer: C

Exam-Day Thinking Framework

When stuck, ask:

Question 1:

What am I trying to do?

Goal	Service
Chat	Azure OpenAI
Company Knowledge	AI Search + OpenAI
Images	AI Vision
Documents	Document Intelligence
Speech	AI Speech
Text Analysis	AI Language
Agentic Work	Foundry Agent

Question 2:

Can deterministic AI solve this cheaper than an LLM?

Examples:

Task	Best Choice
Entity Extraction	Azure Language
Sentiment	Azure Language
OCR	Vision
Invoice Parsing	Document Intelligence
Current Time	Tool Call
Company Docs Q&A	RAG
Open-ended Conversation	LLM

Question 3:

Who is doing what in MCP?

Component	Role
Agent	Thinks
MCP Client	Requests
MCP Protocol	Standard
MCP Server	Exposes tools

Component	Role
Tool	Performs action

That alone will answer a surprising number of AI-901 questions and establishes the mental bridge into AI-102/AI-103 and Foundry agent architecture.

A newer AI-901 / AI-103 exam question I've been asking myself:

"What is the lowest-cost component that can reliably answer the question?"

Execution plan order:

Plain Text
1. Hard-coded logic
2. Filter
3. Search
4. Deterministic AI Service
5. Small Model
6. Large Model
7. Agent Orchestration

Examples:

Plain Text
What time is it?
→ Code

Plain Text
What's the sentiment?
→ Azure Language
Plain Text
Find policy XYZ
→ Search
Plain Text
Summarize this report
→ LLM
Plain Text
Find report → summarize it → email team
→ Agent

That's really the bridge between:

Plain Text

POPULAR C.R.U.D.

and

Plain Text
MCP + Agents + Foundry

Everything becomes:

Plain Text
Input
↓
Transform
↓
Decision
↓
Action
↓
Output

The tools just get progressively more sophisticated.

I actually think your growing glossary could become a useful student reference:

Plain Text
Token = Billable text unit
Embedding = Numerical meaning
Vector = Mathematical representation
Inference = Using a model
Attention = What matters most
RAG = Search before inference
Grounding = Trustworthy context
MCP = Standardized tool access
Agent = Planner/orchestrator
Tool = Action executor

That's far closer to how practitioners think than the traditional certification definitions. The cert teaches **what the services are**. Your notes are starting to explain **why they exist and when to use them**. 👍

Reference

1. Study guide for Exam AI-103: Developing AI Apps and Agents on Azure | Microsoft Learn
2. Course AI-901T00-A: Introduction to AI in Azure - Training | Microsoft Learn
3. Exam AI-901: Microsoft Azure AI Fundamentals - Certifications | Microsoft Learn
4. Study guide for Exam AI-103: Developing AI Apps and Agents on Azure | Microsoft Learn
5. https://learn.microsoft.com/en-us/training/courses/ai-103t00
6. https://learn.microsoft.com/en-us/training/courses/ai-901t00
7. MicrosoftLearning/mslearn-ai-fundamentals: Azure AI Fundamentals exercises